Current Services vs AWS Services

Purpose	Currently Using	Why We Use It Now	AWS Service
Database	Aiven PostgreSQL	Stores application data such as employees, attendance, leave, payroll, etc.	**Amazon RDS for PostgreSQL**
File Storage	Backblaze B2	Stores uploaded files such as PDF, PNG, JPG, etc.	**Amazon S3**
Email	Gmail SMTP	Sends password-reset emails and notifications	**Amazon SES** *(optional; Gmail can continue)*
Application Hosting	Docker / Local Server	Runs the Spring Boot application	**Amazon ECS Fargate**
Docker Image Storage	Local Docker image	Stores the image used to run the application	**Amazon ECR**
Networking	Local/Existing network	Allows application components to communicate	**Amazon VPC + Security Groups**
Secrets	Application configuration/ environment variables	Stores DB password, JWT secret, SMTP credentials, etc.	**AWS Secrets Manager**
Logs & Monitoring	Application/console logs	Used to check errors and application activity	**Amazon CloudWatch**
API Access	Application running on port 8080	Allows frontend/client to call the backend API	**Application Load Balancer (ALB)** *(when exposing API publicly)*
HTTPS/SSL	Not specified in current setup	—	**AWS Certificate Manager (ACM)**
DNS/Domain	Not specified	—	**Amazon Route 53** *(optional)*

Main Changes

1. Aiven PostgreSQL → Amazon RDS PostgreSQL
The database will move from Aiven to AWS RDS. RDS will continue to provide PostgreSQL, so the application does not need to change its database technology.

2. Backblaze B2 → Amazon S3
The application already uses the AWS S3 SDK and `S3Presigner`. Therefore, moving to Amazon S3 is relatively straightforward because S3 can replace the current S3-compatible B2 storage.

3. Gmail SMTP → Amazon SES
Currently Gmail SMTP sends password-reset emails and notifications. We can either continue using Gmail or move to Amazon SES for AWS-based email delivery. SES is therefore **optional**, not mandatory for the initial deployment.

4. Docker Application → ECS Fargate
The Spring Boot application already has a Dockerfile. We can build the Docker image, push it to ECR, and run it using ECS Fargate. This is the recommended AWS deployment approach in the guide.

AWS Architecture

Frontend → ALB → ECS Fargate → RDS PostgreSQL

Supporting services:

ECR → Docker images
S3 → File storage
Secrets Manager → Passwords/secrets
CloudWatch → Logs/monitoring
VPC + Security Groups → Networking
ACM → HTTPS/SSL

In Short

Today	After AWS Migration
Aiven PostgreSQL	**RDS PostgreSQL**
Backblaze B2	**S3**
Gmail SMTP	**SES** *(optional)*
Docker/local hosting	**ECS Fargate**
Local Docker image	**ECR**
Existing networking	**VPC + Security Groups**
Environment secrets	**Secrets Manager**
Console/application logs	**CloudWatch**
Direct API access	**ALB**
No specified SSL setup	**ACM**